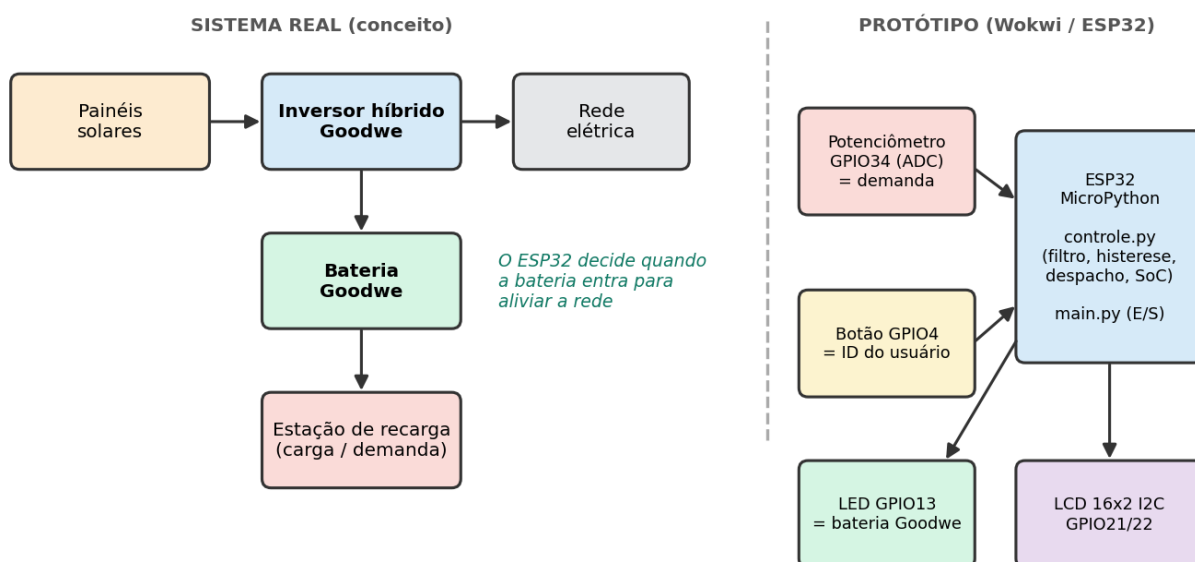


ChargeGrid FIAP

Estação de recarga inteligente com despacho automático entre energia solar, bateria Goodwe e rede elétrica

Arquitetura do ChargeGrid FIAP: sistema real (Goodwe) x protótipo simulado



Telemetria pela Serial (CSV): demanda, solar, bateria, rede, SoC, autonomia e CO2

Integrante	RM
Eduardo Barcelos De Carvalho Brasileiro	573274
Julia Johanson Peniche Dias Da Silva	572220
Lucas Bomfim Leite	570420

Simulação Wokwi: <https://wokwi.com/projects/476802909288083457>

1. Visão geral e problema

A eletrificação da mobilidade cria picos de demanda localizados: vários veículos carregando ao mesmo tempo pressionam a rede elétrica e, sem gestão, esse pico é atendido por energia da rede, nem sempre a mais limpa nem a mais barata. O **ChargeGrid FIAP** é um protótipo de estação de recarga em que um controlador (ESP32) decide, em tempo real, como atender a demanda: **primeiro a energia solar, depois a bateria de armazenamento Goodwe e, por último, a rede.**

O sistema autentica a sessão de recarga (simulada por um botão de “ID”), mede a demanda (simulada por um potenciômetro), aciona a bateria apenas quando necessário e informa ao usuário, no display, a demanda, o estado da bateria, o uso da rede e o CO₂ evitado. Os mesmos dados são emitidos pela porta serial em formato CSV, permitindo análise e visualização posteriores.

2. Alinhamento com o desafio Goodwe e com a disciplina

Ponto do desafio	Como o ChargeGrid atende
Geração, armazenamento e uso de energia renovável com monitoramento inteligente	Modelo de despacho com solar (6 kW), bateria (10 kWh, 5 kW) e rede; o estado de carga (SoC) é calculado e exibido a cada ciclo.
Tecnologia compatível com Goodwe	O LED/estado “BAT ON” representa o acionamento da bateria do sistema híbrido Goodwe; a lógica de decisão corresponde ao que um inversor híbrido com bateria executa (priorizar solar, usar a bateria nos picos, reservar carga mínima).
Algoritmos inteligentes e automações	Média móvel para filtrar ruído, histerese para evitar liga/desliga repetido, reserva mínima de 20% de SoC e despacho por prioridade.
Interface amigável e visualização de dados	Display LCD com duas telas alternadas e telemetria CSV pela Serial, com gráficos gerados a partir dos dados.
Impacto: eficiência, carbono, relevância social	Autonomia renovável, redução do pico de demanda na rede e estimativa de CO ₂ evitado por sessão (seção 7).

Conexão com a disciplina: o projeto integra sensoriamento analógico (ADC), entrada digital com debounce, comunicação I2C, programação em MicroPython, modelagem de sistemas de energia e simulação com Wokwi, conteúdos trabalhados ao longo das sprints.

3. Arquitetura e integração dos sistemas

O protótipo é composto por um ESP32 DevKit C v4 com MicroPython e quatro periféricos. A tabela resume cada conexão, conforme o arquivo `diagram.json` do Wokwi.

Componente	Função no sistema	Pino ESP32	Observação
Potenciômetro	Simula a demanda da estação (0–11 kW)	GPIO34 (ADC)	VCC em 3V3; atenuação 11 dB (0–3,3 V)
Botão (verde)	Simula a leitura do ID: inicia/encerra a sessão	GPIO4	PULL_UP interno; ativo em nível baixo
LED vermelho + 220 Ω	Indica bateria Goodwe acionada	GPIO13	Resistor limita a corrente
LCD 16x2 I2C (0x27)	Interface com o usuário	SDA 21 / SCL 22	Alimentado em 5V; I2C a 400 kHz

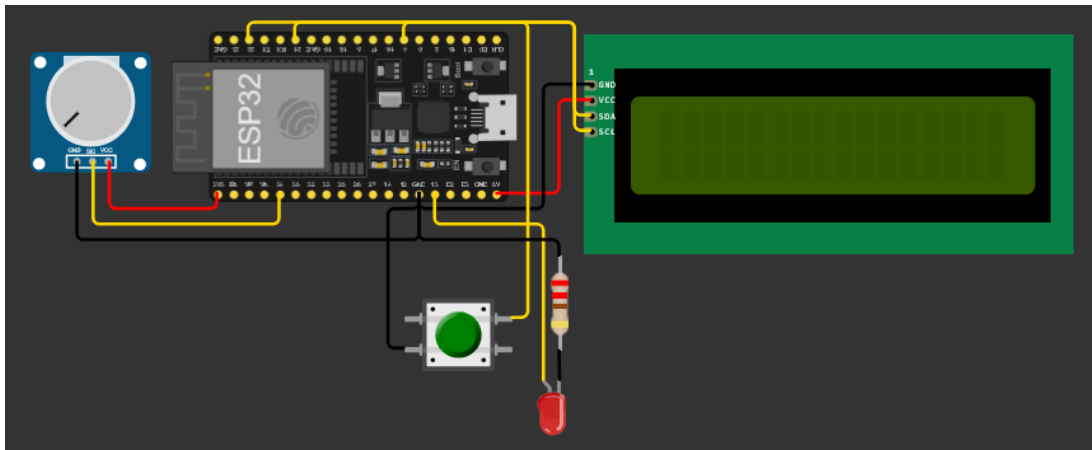


Figura 1 – Circuito real montado no Wokwi (ESP32, potenciômetro, botão, LED com resistor e LCD I2C).

4. Software e algoritmos

O código foi dividido em dois módulos, o que facilita testes e manutenção:

- **main.py** – entrada e saída: leitura de botão (com debounce) e ADC, controle do LED, telas do LCD e telemetria serial.
- **controle.py** – lógica de controle sem dependência de hardware (classe Controlador), executável também em Python comum, o que permitiu simular o comportamento fora do Wokwi.

Em cada ciclo de 500 ms, o controlador executa quatro etapas:

- **Filtragem:** média móvel de 8 amostras sobre a leitura do ADC, que suaviza ruído e oscilações bruscas.
- **Decisão com histerese:** a bateria liga acima de 2500 ($\approx 61\%$ do curso, $\approx 6,7$ kWh) e só desliga abaixo de 2200 ou ao atingir a reserva de 20% de SoC. Isso evita que o LED “pisque” quando a demanda oscila perto do limite.
- **Despacho por prioridade:** solar atende a demanda primeiro; o excedente carrega a bateria; o que faltar é coberto pela bateria (se ativa) e, por fim, pela rede.
- **Contabilização:** integração de energia (kWh) de solar, bateria e rede, SoC, autonomia renovável (%) e CO₂ evitado. Um fator de aceleração (1 s real = 2 min simulados) torna a variação do SoC visível na simulação.

Fluxograma do controle (a cada ciclo de 500 ms)

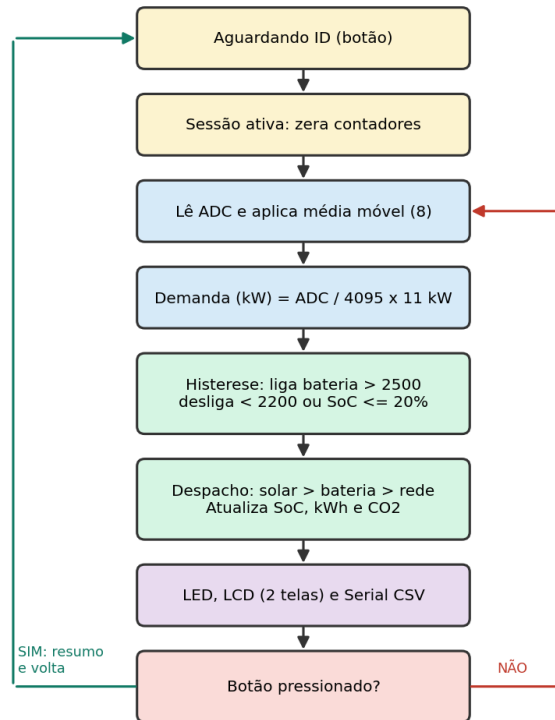


Figura 2 – Fluxograma do ciclo de controle.

5. Interface e experiência de uso

Momento	Linha 1 do LCD	Linha 2 do LCD
Aguardando ID	ChargeGrid FIAP	Aguardando ID...
Início da sessão (1,5 s)	Sistema Ativo!	ID lido (botao)
Tela 1 (a cada 3 s)	Dem: 66% 7.3kW	BAT ON SoC: 84%
Tela 2 (alterna com a 1)	Rede 0.0 Sol 6.0	CO2 evit 0.05kg
Fim da sessão	Sessao encerrada	Autonomia 99%
Resumo final	CO2 evit 0.120kg	Pico bat 3.5kW

Além do display, a Serial emite um registro CSV por ciclo (t_s , adc , $demanda_kw$, $solar_kw$, $bateria_kw$, $rede_kw$, bat_on , soc_pct , $autonomia_pct$, $co2_kg$), que pode ser copiado para uma planilha ou lido por um script para gerar gráficos.

6. Resultados obtidos

Resultados qualitativos (simulação no Wokwi). A sessão só inicia após o botão; o display mostra “Rede/Solar” enquanto a demanda está abaixo do limite; ao girar o potenciômetro além de $\approx 61\%$ do curso, o LED acende e o display passa a indicar “BAT ON”; ao reduzir a demanda abaixo de $\approx 54\%$ (2200) a bateria desliga. Pressionar o botão novamente encerra a sessão e exibe o resumo.

Resultados quantitativos. Para medir o ganho do algoritmo, o controlador foi executado com um perfil de demanda reproduzível (o potenciômetro sobe de $\approx 25\%$ a $\approx 95\%$ e retorna, em 90 s, equivalentes a 3 h simuladas). O script `simulacao/simular_perfil.py` gera os dados e o gráfico abaixo.

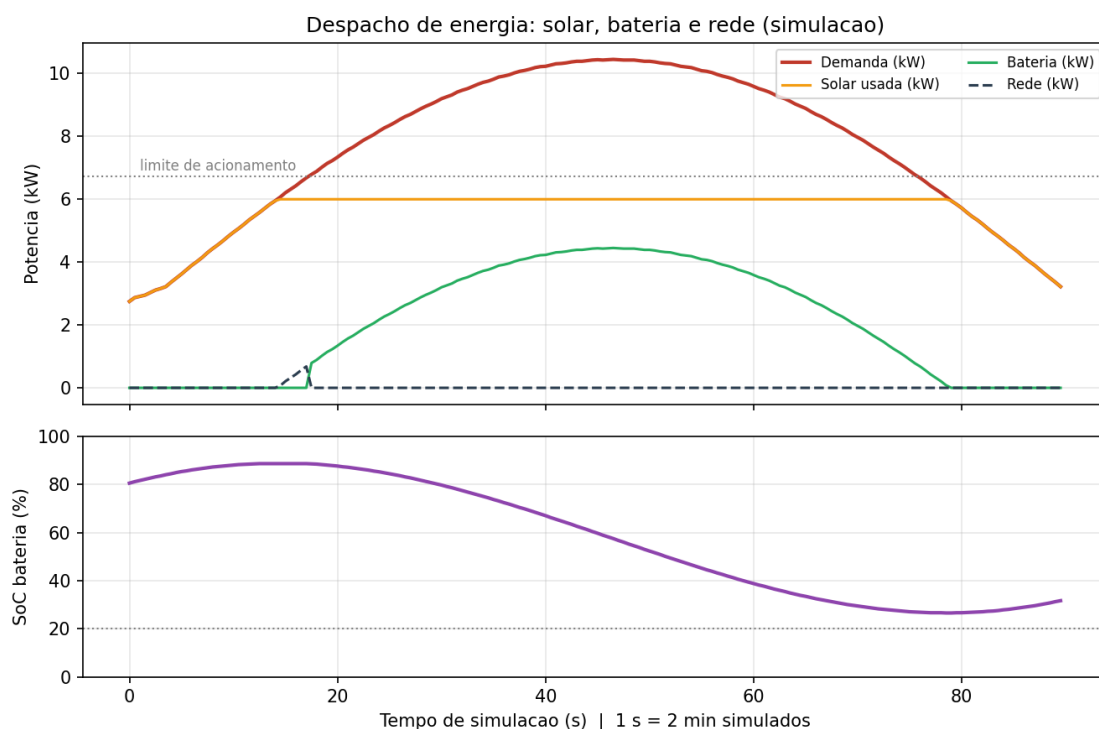


Figura 3 - Despacho entre solar, bateria e rede e evolução do SoC no perfil simulado.

Indicador	Valor (perfil simulado)
Energia demandada pela estação	22,86 kWh
Atendida por solar / bateria / rede	16,63 kWh / 6,20 kWh / 0,04 kWh
Autonomia renovável (solar + bateria)	99,8%
Pico de demanda	10,45 kW
Pico na rede sem bateria (apenas solar)	4,45 kW
Pico na rede com a lógica ChargeGrid	0,68 kW (redução de 84,7%)
SoC da bateria: início → fim	80% → 31,7% (reserva mínima de 20% preservada)
CO ₂ evitado (premissa de 0,0385 kg/kWh)	0,88 kg

Observação: os valores de potência, capacidade e fator de emissão são premissas de simulação, ajustáveis em controle.py; não são medições de um equipamento real.

7. Impactos: viabilidade, eficiência e sustentabilidade

- **Viabilidade técnica:** a solução usa um microcontrolador de baixo custo e lógica simples, replicável em um gateway real que leia dados do inversor (por exemplo, via Modbus ou API de monitoramento).
- **Eficiência:** priorizar solar e usar a bateria só nos picos reduz a potência exigida da rede (−84,7% no perfil simulado) e preserva a vida útil da bateria com reserva mínima e histerese.
- **Carbono:** cada kWh renovável usado no lugar de energia da rede evita emissões; o sistema contabiliza essa estimativa por sessão e a mostra ao usuário.
- **Relevância social:** estações que aliviam a rede facilitam a expansão da mobilidade elétrica e tornam visível ao usuário o consumo de energia limpa.

8. Avaliação crítica

Benefícios e inovação: separação entre controle e hardware, algoritmo com filtro, histerese e reserva de bateria, telemetria de dados e métricas de sustentabilidade por sessão. **Limitações:** demanda e ID são simulados (potenciômetro e botão); geração solar é constante; não há integração real com equipamentos Goodwe; o fator de CO_2 é uma premissa. **Próximos passos:** ler dados reais do inversor, usar previsão de geração e tarifa horária, autenticação RFID/QR, envio MQTT para dashboard e integração com assistente virtual.

9. Tecnologias e referências

Item	Uso no projeto
ESP32 DevKit C v4	Microcontrolador (ADC, I2C, GPIO)
MicroPython 1.21 (build micropython-20231005)	Linguagem e firmware do ESP32
Wokwi	Simulação do circuito e do firmware
LCD 1602 com adaptador I2C; bibliotecas lcd_api / machine_i2c_lcd	Interface com o usuário
Potenciômetro, botão, LED, resistor 220 Ω	Demanda, ID, indicação da bateria, proteção do LED
Python 3 + matplotlib	Simulação do controlador e geração de gráficos
GitHub	Versionamento e documentação
Goodwe (inversores híbridos e baterias de armazenamento)	Tecnologia de referência do desafio

Simulação: <https://wokwi.com/projects/476802909288083457>